

SQL DDL/DML Challenge:

Erstellung einer Datenbank für eine Schule

Eine Schule möchte ihre Daten in einer Datenbank verwalten. Dabei sollen folgende Aspekte berücksichtigt werden:

- **Lehrer:** Jeder Lehrer hat eine eindeutige ID, einen Nachnamen und einen Vornamen. Ein Lehrer kann von keiner, einer oder mehreren Klassen der Klassenlehrer sein und er kann keine, eine oder mehrere AGs leiten
- **Klassen:** Die Schule hat mehrere Klassen. Jede Klasse hat eine eindeutige ID, einen Namen (z.B. "8a") und immer genau einen Klassenlehrer, der durch seine Lehrer-ID referenziert wird. Eine Klasse hat mehrere Schüler
- **Schüler:** Jeder Schüler hat eine eindeutige ID, einen Nachnamen, einen Vornamen und gehört immer zu genau einer bestimmten Klasse. Die Klassenzugehörigkeit wird durch die Klassen-ID referenziert.
- **Arbeitsgemeinschaften (AGs):** Die Schule bietet verschiedene Arbeitsgemeinschaften an. Jede AG hat eine eindeutige ID, einen Namen (z.B. "Fußball-AG") und wird immer von genau einem Lehrer geleitet. Der Leiter wird durch seine Lehrer-ID referenziert. Es können 0 bis n Schüler an einer AG teilnehmen.
- **Teilnahme an AGs:** Schüler können an keiner, einer oder mehreren AGs teilnehmen. Es soll erfasst werden, welcher Schüler an welcher AG teilnimmt.

Aufgaben:

1. Entwerfen Sie ein **relationales Datenmodell**, das die oben genannten Aspekte der Schuldatenbank abbildet. Sie können den fachlichen Sachverhalt zunächst in einem ER-Modell abbilden um dann daraus das relationale Datenmodell ableiten (optional, aber empfohlen)
2. Erstellen Sie **SQL DDL (Data Definition Language) Statements**, um die entsprechenden Tabellen in einer Datenbank zu definieren. Achten Sie dabei auf:
 - Geeignete Datentypen für die einzelnen Attribute.
 - Primärschlüssel für jede Tabelle, um eine eindeutige Identifizierung der Datensätze zu gewährleisten.
 - Fremdschlüsselbeziehungen, um die Beziehungen zwischen den Tabellen (z.B. zwischen Schüler und Klasse, Lehrer und Klasse, Schüler und AG) herzustellen und die referenzielle Integrität zu gewährleisten.

****Zusätzliche Hinweise****

- * Überlegen Sie, welche Beziehungen zwischen den Entitäten (Lehrer, Klasse, Schüler, AG) bestehen (z.B. 1:n, n:m).
- * Berücksichtigen Sie, dass ein Lehrer mehrere Klassen unterrichten und mehrere AGs leiten kann.
- * Ein Schüler kann nur einer Klasse angehören, aber an mehreren AGs teilnehmen.
- * Verwenden Sie sprechende Namen für Tabellen und Attribute.

3. Befüllen sie die Tabellen mit Daten (SQL DML)

Insert Skripte mit Beispieldaten (diese vorgeben !!)

AG

id	name	lehrer_id
3	Theater	2
4	Orchester	1
5	Homepage	3
6	Bigband	1
7	Holzwerken	3
8	Fotografie	3

klasse

id	name	klassenlehrer_id
1	8A	2
2	8B	4
3	8C	1
4	8D	3

lehrer

id	name	vorname
1	Buttenmüller	Georg
2	Zimmermann	Josef
3	Amann	Brigitte
4	Huber	Erika
5	Rees	Günter

Schueler

id	name	vorname
1	Meier	Johannes
2	Schwarz Müller	Maria
3	Schmidt	Michael
4	Ebert	Anne
8	Zimmermann	Anne
9	Wiesenhoff	Christian

teilnahmeag

schueler_id	ag_id
1	7
1	8
2	4
2	6
3	3
3	4
3	7
4	5
4	8

4. Machen Sie Anpassungen/Erweiterungen in der Datenbank (SQL DDL)

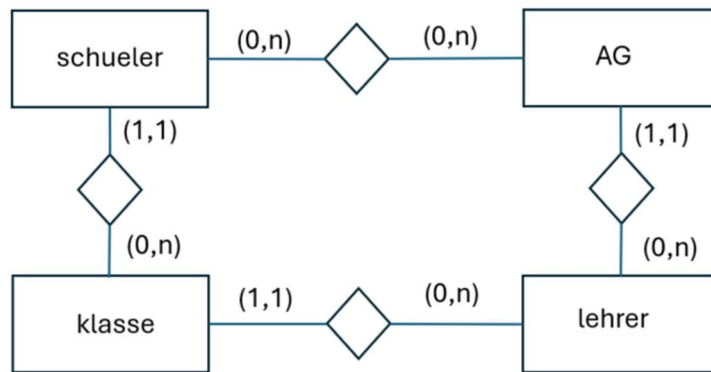
- Ergänzen sie die Tabelle „schueler“ um ein weiteres Attribut „e-mail“. Dieses Attribut soll einen Text mit maximal 50 Zeichen zulassen. Der DEFAULT-Wert für dieses Attribut ist mit „noemail“ zu definieren
- Ergänzen sie die Tabelle „AG“ um ein weiteres Attribut „Wochenstunden“. Als Eingabe sind Ganzzahlen erlaubt. Schränken sie den gültigen Wertebereich auf Werte zwischen 2 und 8 ein.
- Definieren Sie in der Tabelle „AG“ das Attribut „Name“ als Pflicht feld, d.h. NULL ist nicht erlaubt für dieses Attribut

5. Löschen sie die Tabellen. Beachten sie dabei die notwendige Reihenfolge aufgrund der referentiellen Integrität

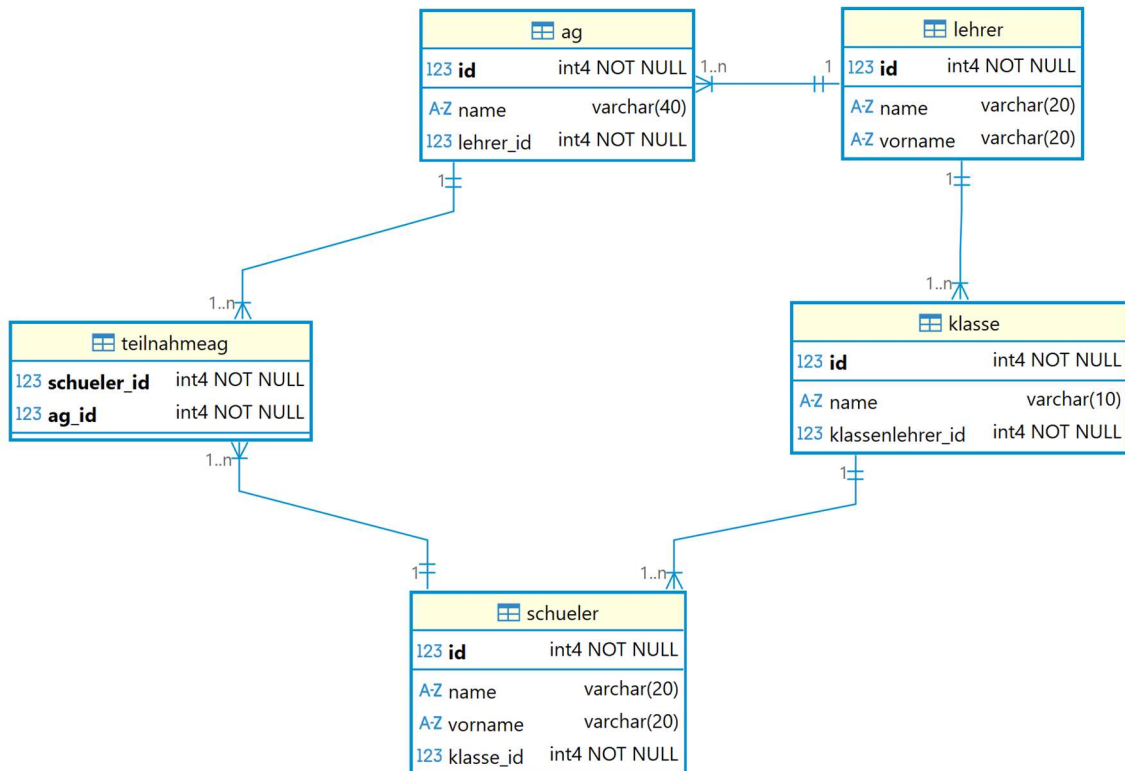
Lösungen

1) Relationales Datenmodell

Optional: ER-Modell (mit Kardinalitäten!), ohne Attribute (optional)



Relationenschema:



2) DDL Befehle

Information zu Beziehungen aus dem Text ableiten:

Beziehung Klasse - Lehrer

- Jede Klasse hat**immer genau einen Klassenlehrer**
 - Der PK „id“ der Tabelle „lehrer“ ist als FK in der Tabelle „klasse“
(wegen „einen“ – eine Klasse hat damit nur eine Referenz zu einem Lehrer--
--- Zusätzlicher Hinweis: Wenn „mehrere“ möglich wären, dann müsste man eine Beziehungsrelation definieren)
 - NOT NULL wegen „immer ...“
- „Ein Lehrer kann von ... **mehreren Klassen** der Klassenlehrer sein ...“
 - Ist erfüllt, da „id“ von der tabelle „lehrer“ als FK in Tabelle „klasse“ referenziert ist und nicht unique ist. D.h. der gleiche Lehrer kann mehrfach referenziert werden

Beziehung AG - Lehrer

- Jede AG .. wird **immer von genau einem Lehrer** geleitet.
 - Der PK lehrer(ID) ist in der Tabelle AG FK. (wegen „einen“)
 - NOT NULL wegen „immer“
- „Ein Lehrer kann ... **mehrere AGs leiten** ...“

- Ist erfüllt, da Lehrer-ID als FK in Tabelle AG ist und nicht unique ist. D.h. der gleiche Lehrer kann mehrfach referenziert werden

Beziehung Klasse - Schüler

- „Jeder Schüler ... gehört **immer zu genau einer** bestimmten Klasse...“
 - => PK „id“ der Tabelle „klasse“ ist als FK in Tabelle schueler. (wegen „einer“)
 - => NOT NULL wegen „immer“
- „Eine Klasse hat **mehrere Schüler.**“
 - Ist erfüllt, da PK „id“ der Tabelle „klasse“ FK in Tabelle „schueler“ ist und nicht unique ist. D.h. die gleiche Klasse kann durch mehrere Schüler referenziert werden
 - er gleiche Lehrer kann mehrfach referenziert werden

Beziehung Schüler - AG

- „... Es können 0 **bis n Schüler** an einer AG teilnehmen...“ „... Schüler können an **mehreren AGs** teilnehmen....“
 - m:n Beziehung über Beziehungsrelation

```
CREATE SCHEMA schule;
SET SCHEMA 'schule';
```

```
/* Reihenfolge der Tabellenanlage beachten (FK-
Abhängigkeit
*/
```

```
CREATE TABLE lehrer(
  id int,
  name varchar(20),
  vorname varchar(20),
  PRIMARY KEY (id)
);
```

```
CREATE TABLE klasse(
  id int,
  name varchar(10),
  klassenlehrer_id int NOT NULL,
  PRIMARY KEY (id),
```

da FK mandatory

-- NOT NULL

```

    FOREIGN KEY (klassenlehrer_id) REFERENCES lehrer(id)
);

CREATE TABLE schueler(
    id int,
    name varchar(20),
    vorname varchar(20),
    klasse_id int NOT NULL,          -- NOT NULL da FK
mandatory
    PRIMARY KEY (id),
    FOREIGN KEY (klasse_id) REFERENCES klasse(id)
);

CREATE TABLE ag(
    id int,
    name varchar(40),
    lehrer_id int NOT NULL,         -- NOT NULL da FK
mandatory
    PRIMARY KEY (id),
    FOREIGN KEY (lehrer_id) REFERENCES lehrer(id)
);

CREATE TABLE teilnahmeag(
    schueler_id int,
    ag_id int,
    PRIMARY KEY (schueler_id, ag_id),
    FOREIGN KEY (ag_id) REFERENCES ag(id),    -- NOT NULL
implizit durch PK
    FOREIGN KEY (schueler_id) REFERENCES schueler(id)    --
NOT NULL implizit durch PK
);

```

3) Insert-Skripte für die Befüllung der Tabellen (SQL DML)

```

INSERT INTO lehrer (id, name, vorname) VALUES
(1, 'Buttenmüller', 'Georg'),

```

```
(2, 'Zimmermann', 'Josef'),  
(3, 'Amann', 'Brigitte'),  
(4, 'Huber', 'Erika'),  
(5, 'Rees', 'Günter');
```

```
INSERT INTO klasse (id, name, klassenlehrer_id) VALUES  
(1, '8A', 2),  
(2, '8B', 4),  
(3, '8C', 1),  
(4, '8D', 3);
```

```
INSERT INTO schueler (id, name, vorname, klasse_id) VALUES  
(1, 'Meier', 'Johannes', 2),  
(2, 'Schwarz Müller', 'Maria', 3),  
(3, 'Schmidt', 'Michael', 1),  
(4, 'Ebert', 'Anne', 2),  
(8, 'Zimmermann', 'Anne', 4),  
(9, 'Wiesenhoff', 'Christian', 2);
```

```
INSERT INTO ag (id, name, lehrer_id) VALUES  
(3, 'Theater', 2),  
(4, 'Orchester', 1),  
(5, 'Homepage', 3),  
(6, 'Bigband', 1),  
(7, 'Holzwerken', 3),  
(8, 'Fotografie', 3);
```

```
INSERT INTO teilnahmeag (schueler_id, ag_id) VALUES  
(1, 7),  
(1, 8),  
(2, 4),  
(2, 6),  
(3, 3),  
(3, 4),  
(3, 7),  
(4, 5),  
(4, 8);
```

4) Änderung Tabellendefinitionen (SQL DDL)

```
-- DDL für Tabellenänderungen
```

```

-- Hinzufügen email als Attribut
ALTER TABLE schueler
ADD COLUMN email varchar(50) DEFAULT 'noemail';

-- Hinzufügen attribut wochenstunden mit Wertebereich
ALTER TABLE ag
ADD COLUMN wochenstunden int CHECK(wochenstunden BETWEEN 2
und 8);

-- Alternativ: zuerst Spalte hinzufügen, dann CONSTRAINT
ALTER TABLE ag
ADD COLUMN wochenstunden int;
ALTER TABLE ag
ADD CONSTRAINT wochenstundenanzahl CHECK(wochenstunden
BETWEEN 2 and 8);

-- Ändern Spaltendefinition auf NOT NULL
ALTER TABLE ag
ALTER COLUMN name SET NOT NULL;

```

5) Löschen der Tabellen

-- Löschen der Tabellen. Reihenfolge beachten. Zunächst nur Tabellen deren PK nicht als FK referenziert werden

```

DROP TABLE teilnahmeag;
DROP TABLE schueler;
DROP TABLE klasse;
DROP TABLE AG;
DROP TABLE lehrer;

```